

Attention Quantization in Video Diffusion Models

程煜格 20260612

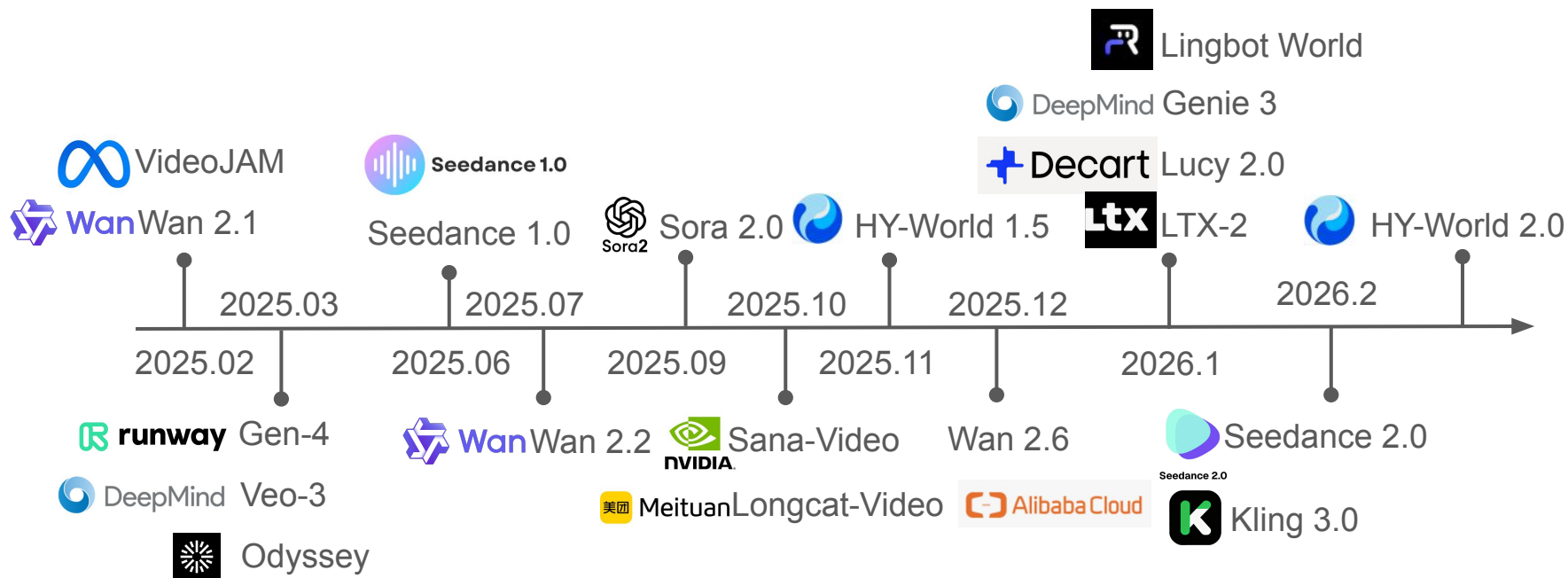
Agenda

- Why attention quantization matters in video diffusion models?
- Why attention quantization is challenging?
- Representative Approaches
 - [ICLR 25] SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration
 - [ICML 25] SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization
 - [NeurIPS 25] PAROAttention: Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models
- Conclusion and discussion

REAL OR AI ?



Video Generation Models Timeline



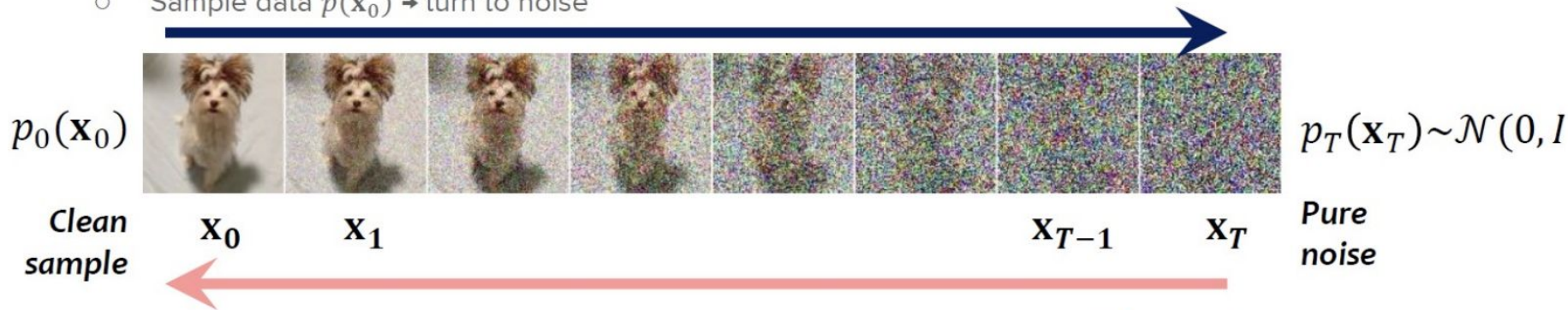
Background: diffusion model

Forward process (train): Gradually add gaussian noise of different levels

Backward process (inference): Gradually denoise the gaussian noise

- Intuition: the NN learns to predict the “noise” at each timestep.

- Sample data $p(\mathbf{x}_0) \rightarrow$ turn to noise

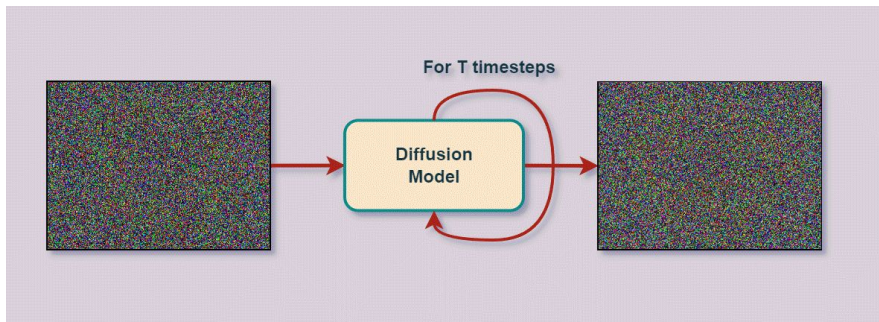


- **Reverse / denoising process**

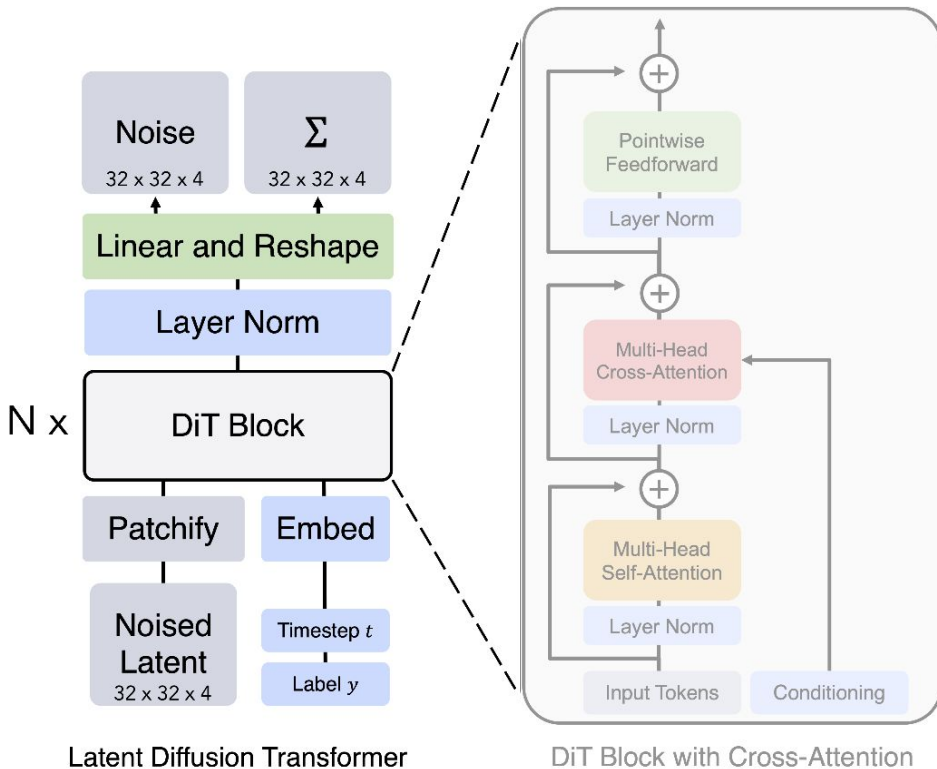
Background: video diffusion inference pipeline

Inference diffusion process

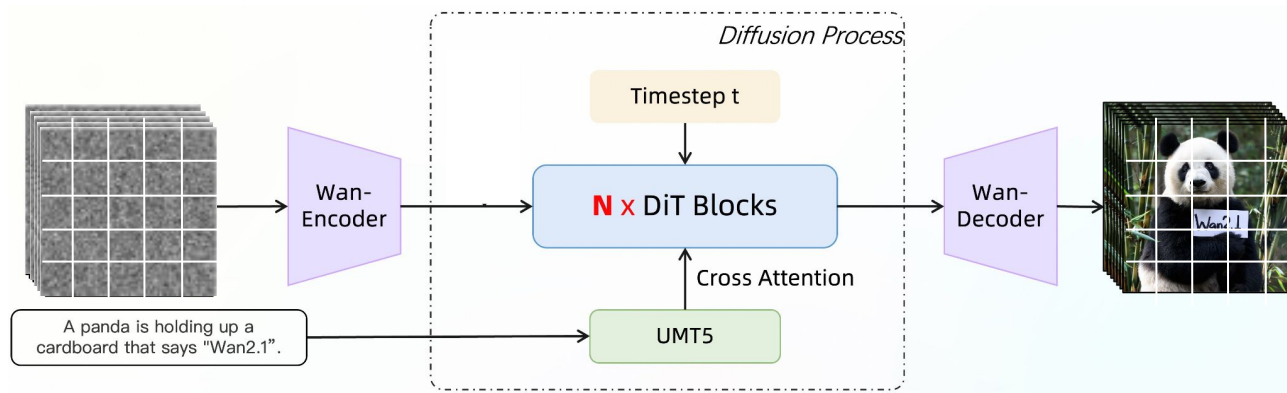
- Denoise noised tokens by $T \times$ timesteps



- In each step: **N x DiT Blocks**
 - a. Self-attention
 - b. Cross-attention
 - c. MLP/FFN



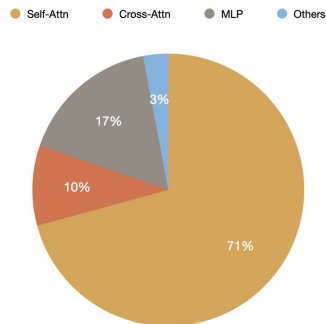
Background: video diffusion inference pipeline



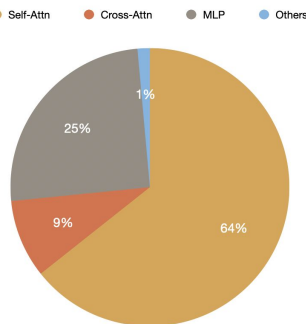
Attention is the bottleneck in video diffusion!

Inference diffusion process is **slow**

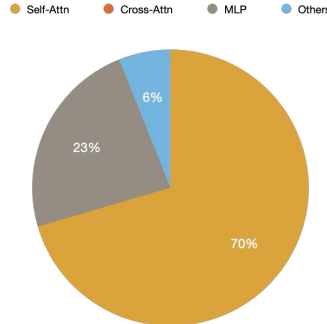
- It takes ~30mins for Hunyuan Video (13B) to generate a 5s-1280p video on H100
- **and self-attention dominated**



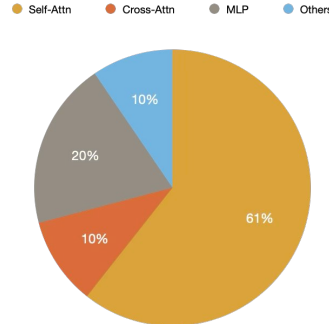
Wan2.1



Turbo Diffusion



Hunyuan Video

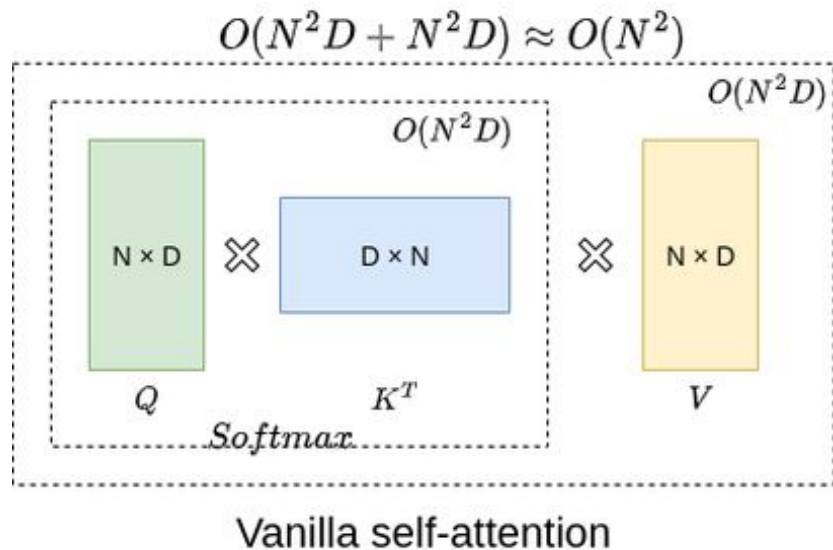
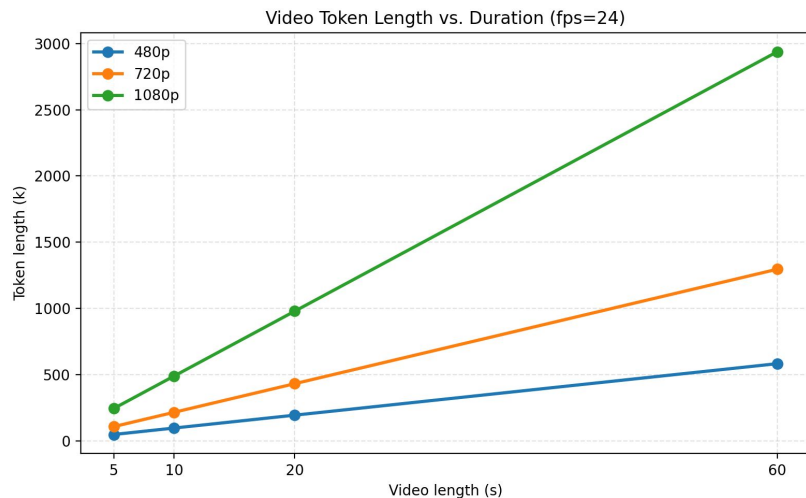


Longcat-Video

Attention is the bottleneck ... why?

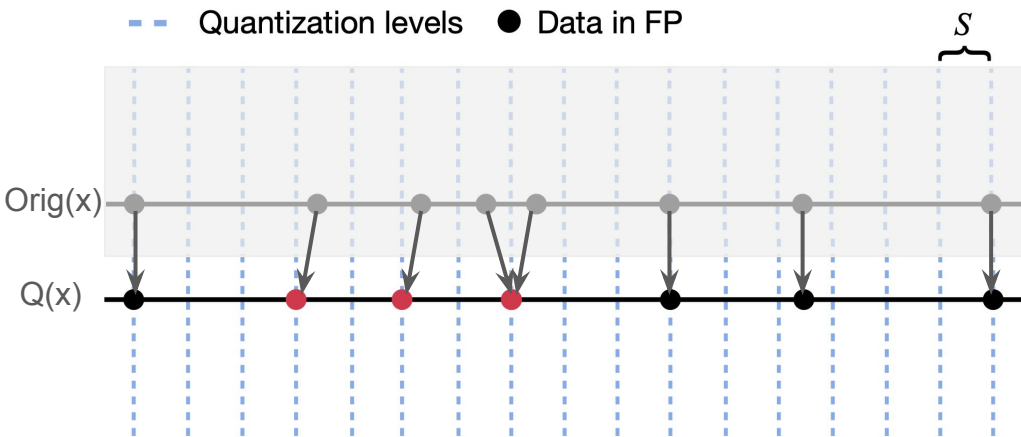
Self-Attention complexity: $O(N^2)$, #token $N=T \times H \times W$

- Generate more frames $T \uparrow \rightarrow \text{token} \uparrow$
- Generate higher resolution $H \times W \uparrow \rightarrow \text{token} \uparrow$



Quantization helps reducing attention mem. & comp.

Quantization maps high-precision floating-point values (x) into low-bit discrete levels $Q(x)$.



$$\mathbf{x}_{\text{int}} = \text{clamp} \left(\left\lfloor \frac{\mathbf{x}}{s} \right\rfloor; -2^{b-1}, 2^{b-1} - 1 \right)$$
$$\hat{\mathbf{x}} = s \mathbf{x}_{\text{int}}$$

- Reduce #bits to store each element
- Use efficient low-bit arithmetic for computation

$$\vec{q} \times \vec{k} = \sum_i q_i \times k_i \approx \sum_i \hat{q}_i \times \hat{k}_i = s_q s_k \sum_i q_i^{(\text{int})} \times k_i^{(\text{int})}$$

Data Type	Throughput
FP32	19.5 TFLOPS
FP16	312 TFLOPS
INT8	624 TOPS

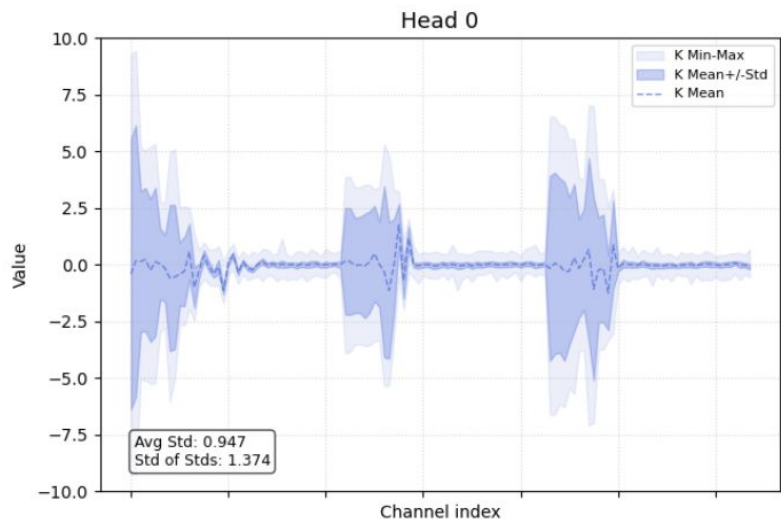
Theoretical throughput summit of NVIDIA A100 tensor core

* TFLOPS (Tera Floating Point Operations Per Second)

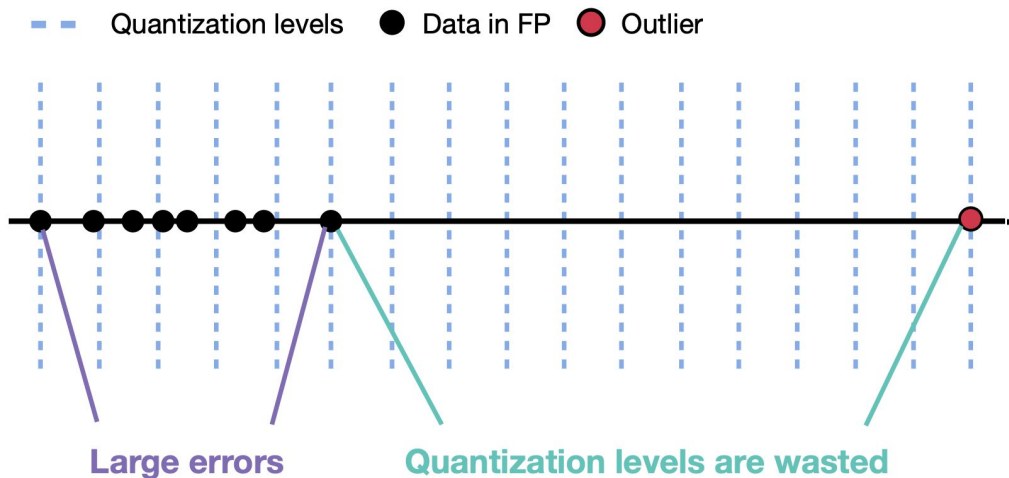
* TOPS (Tera Operations Per Second)

Attention quantization is challenging: (1) outliers

- **Few** activation channels are **consistently larger** than others.
- Outliers **ruin** quantization accuracy



Activations from TurboDiffusion



Attention quantization is challenging: (2) softmax sensitivity

- For a linear layer:

$$Y = XW$$

Quantization usually becomes:

$$Y \approx (Q(X)Q(W))$$

Where

- $Q(\cdot)$ denotes quantization
- errors are mostly linear

- Attention contains three stages:

$$QK^T \rightarrow \textit{Softmax} \rightarrow PV$$

Small quantization error in logits ($Q \cdot K^T$) may become much larger after softmax.

$$\textit{softmax}(x)_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

Attention quantization is challenging: (3) hardware efficiency

Quantization reduces arithmetic precision, but the associated overhead may negate the theoretical speedup.

- QK^T and PV are both activation-activation quantization. Dynamic range may changes
 - every layer | timestep | sample => dynamic quantization are often required at runtime.
- Softmax requires higher precision and causes dequant and requant overhead:

$$QK^T \xrightarrow{\text{dequant}} \text{Softmax} \xrightarrow{\text{requant}} PV$$

- Compatibility with FlashAttention

Therefore, a successful attention quantization method must:

- ✓ preserve attention computation accuracy
- ✓ handle activation outliers
- ✓ delivering hardware acceleration with comparable quality.

SageAttention

Accurate 8-Bit Attention for Plug-and-play Inference Acceleration



Main idea: Figure 1: An example of SageAttention on video generation (CogvideoX on RTX4090).

1. Quantize Q/K to INT8 for fast QK^T
2. Use smoothing to handle outliers.
3. Keep critical parts accurate enough so end-to-end generation quality does not collapse.

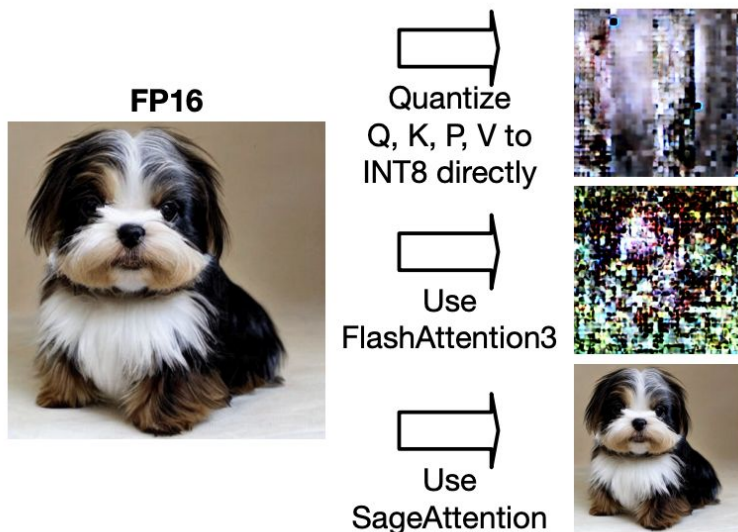
Pros:

- Training-free acceleration
- almost no end-to-end metrics loss across diverse models

SageAttention

INT8 Q/K quantization

Naive 8-bit quantization damages generation.



Why INT8?

- INT8 Matmul are x4 times faster than in FP16 and x2 times faster than FP8.
- INT8 quantization for matrices (Q, K) is more precise than FP8 in attention

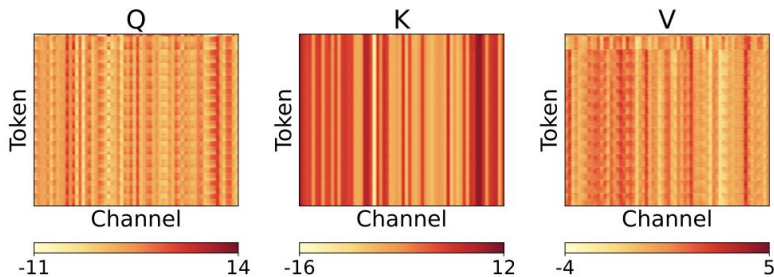
Table 2: **Average accuracy** using different data types across all layers of real models.

Q, K	$\tilde{P}, V$	Cos Sim $\uparrow$	Relative L1 $\downarrow$	RMSE $\downarrow$
INT8	E4M3	99.94%	0.0345	3.53e-3
	E5M2	99.81%	0.0572	6.11e-3
	INT8	99.70%	0.1035	6.82e-3
E4M3	E4M3	99.81%	0.0607	5.93e-3
	E5M2	99.68%	0.0769	7.72e-3
	INT8	99.58%	0.1199	8.31e-3
E5M2	E4M3	99.37%	0.1107	1.09e-2
	E5M2	99.22%	0.1213	1.20e-2
	INT8	99.13%	0.1583	1.24e-2

SageAttention

Two primary challenges in applying INT8 quantization

1. The matrix K exhibits a significant channel-wise outlier
2. Simply quantizing (P, V) into INT8 does not consistently ensure the accuracy of P·V



(a) QKV distribution in Cogvideo

Table 3: **Worst accuracy** using different data types across all layers of real models.

Q, K	$\tilde{P}, V$	Cos Sim $\uparrow$	Relative L1 $\downarrow$	RMSE $\downarrow$
INT8	E4M3	76.36%	0.5899	0.4311
	E5M2	78.98%	0.4233	0.4371
	INT8	56.40%	0.7921	0.5405
	FP16	99.99%	0.0116	0.0091

SageAttention: methodology

1. Smooth K with K-mean to handle channel-wise outliers

Before quantizing: $\gamma(K) = K - \text{mean}(K)$

$$\sigma(q(K - \text{mean}(K)^\top)) = \sigma(qK^\top - q \cdot \text{mean}(K)) = \sigma(qK^\top).$$

Table 1: End-to-end metrics comparison of different quantization methods.

Quantization (Q, K)	Smoothing K	Llama WikiText ↓	CogVideo (Fscore) ↑	Unidiffuser (FID) ↓	UltraPixel (FID) ↓	TIMM ImageNet ↑
Full-Precision	-	5.823	3.768	163.33	179.78	84.79%
Per-token	✗	5.824	1.924	221.18	193.36	84.21%
	✓	5.824	3.734	166.52	179.79	84.74%
Per-block	✗	5.825	2.014	229.08	195.67	84.18%
	✓	5.824	3.718	166.93	179.98	84.76%
Per-tensor	✗	5.826	1.902	267.06	196.26	84.12%
	✓	5.824	3.640	167.65	180.21	84.69%
FlashAttn3 (with quant)		5.850	3.394	394.13	383.61	84.70%

2. maintain (P, V) in FP16 and use a low-precision FP16 accumulator.

Table 4: **Average accuracy** using different accumulators across all layers of real models.

Accum.	Cos Sim ↑	Relative L1 ↓	RMSE ↓
FP32	99.98%	0.0156	2.94e-3
FP16	99.98%	0.0156	2.94e-3

Table 5: **Worst accuracy** using different accumulators across all layers of real models.

Accum.	Cos Sim ↑	Relative L1 ↓	RMSE ↓
FP32	99.84%	0.0511	4.229e-3
FP16	99.84%	0.0511	4.229e-3

SageAttention: evaluation

1. Quality performance

Model	Attention	CLIPSIM $\uparrow$	CLIP-T $\uparrow$	VQA-a $\uparrow$	VQA-t $\uparrow$	FScore $\uparrow$
CogvideoX	Full-Precision	0.1837	0.9976	68.962	75.925	3.7684
	SageAttention	0.1836	0.9976	68.839	75.037	3.8339

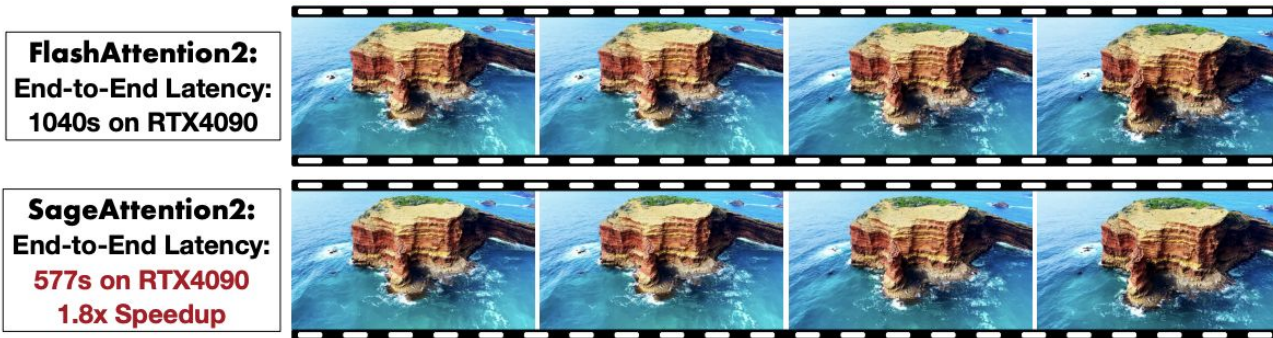
2. Speedup

Table 7: Real speedup of SageAttention on RTX4090.

Model	Shape of Q, K, V	Original attention	SageAttention	Speedup
CogvideoX	(2, 30, 17776, 64)	163.37 (FlashAttn2)	327.57	2.01x

SageAttention2

Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization



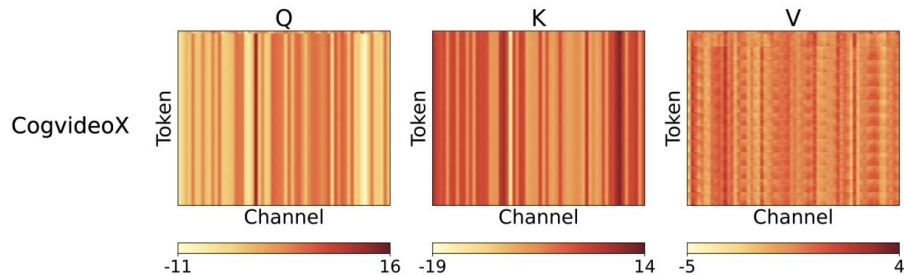
Main idea:

1. Further reduce precision (INT4 / FP8) while maintaining generation quality
 - Q, K $\rightarrow$ INT4 quantization
 - P, V $\rightarrow$ FP8 quantization (E4M3)
2. Two-stage Attention computation
 - Q + K Dual Smoothing
3. Per-thread Quantization: Each GPU thread maintains its own quantization scale

SageAttention2

Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization

1. Q + K Dual Smoothing



$$\gamma(Q_i) = Q_i - \bar{q}_i, \quad \gamma(K_j) = K_j - \bar{k}, \quad (2)$$

where $\bar{q}_i = \text{mean}(Q_i)$, $\bar{k} = \text{mean}(K)$ are $1 \times D$ vectors,

$$S_{ij} = Q_i K_j^\top = (\bar{q}_i + \gamma(Q_i))(\bar{k} + \gamma(K_j))^\top = \bar{q}_i \bar{k}^\top + \bar{q}_i \gamma(K_j)^\top + \gamma(Q_i) \bar{k}^\top + \gamma(Q_i) \gamma(K_j)^\top = \bar{q}_i \bar{k}^\top + \bar{q}_i \gamma(K_j)^\top + \gamma(Q_i) \bar{k}^\top + \Delta S_{ij} + b.$$

2. Per-thread Quantization: Each GPU thread maintains its own quantization scale

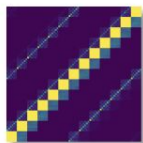
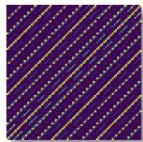
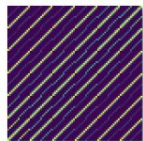
3. Two-level Accumulation for P·V

- Problem: the accumulator for the *mma(f32f8f8f32)* instruction is actually FP22 $O_{i+1} = (P_i \cdot V_i) + O_i$
 - S1E8M13
 - Least significant 10 mantissa bits zeroed out (i.e., truncated)
- Solution: add a real FP32 accumulator

PAROAttention

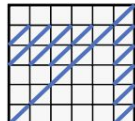
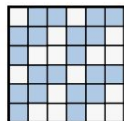
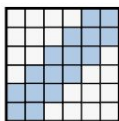
Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models

Challenges:



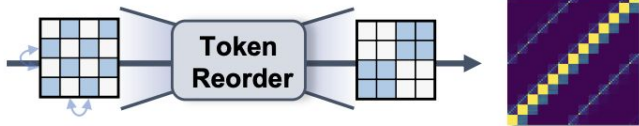
Diverse Visual
Attention Patterns

Existing Approach: *Accommodate* Patterns
With Specialized Sparse Mask Design



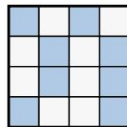
Face challenge under low dense rate

Our Approach: *Reorganize* Attention Patterns



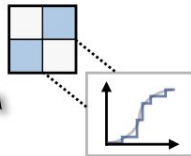
Unify into hardware-friendly block-wise pattern

Compression Techniques



Block-wise
Sparse Attention

Benefits



Block-wise
Quantization

CogVideoX
(720P 6s)



Original Attention (FP16 + Dense)



PAROAttn
(INT8 + 30% Dense)

5.7x Attn Speedup
2.72x E2E Speedup

Flux
(1024x1024)



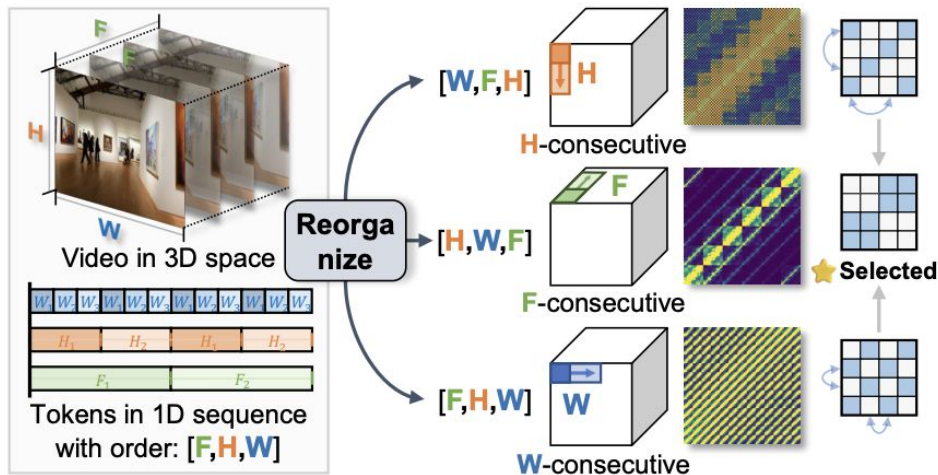
PAROAttn
(INT4 + 75% Dense)

2.5x Attn Speedup
1.94x E2E Speedup

PAROAttention

Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models

- Utilizing reordering tokens to reorganize attention pattern.
Metric for reorder:
 - Local sparsity & local uniform
 - Choose the permute order for each head offline.



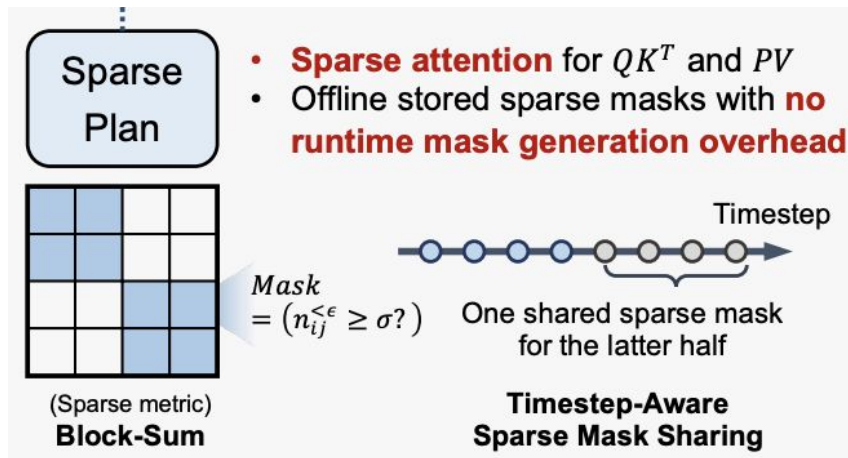
(c) Unify into Block-wise Patterns by Permutation

PAROAttention

Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models

2. How to decide sparse mask?

- Block-wise sparsity.
Skipping block in FlashAttention.
- Offline, base on post-softmax attention map.

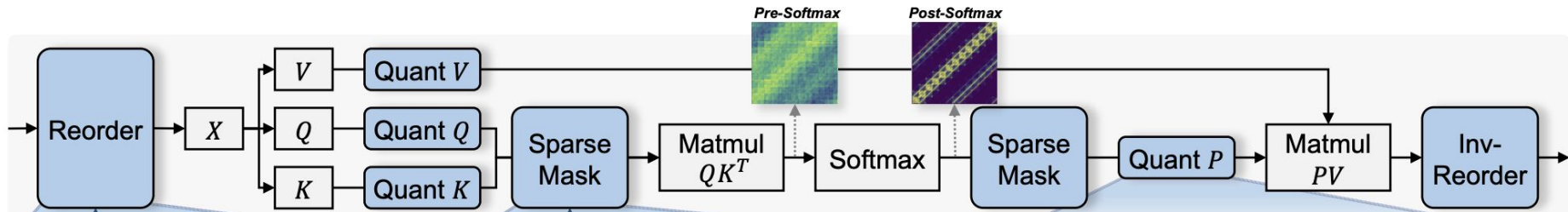


PAROAttention

Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models

3. How to perform quantization?

- Quantize QK^T and PV
- Block-wise uniform distribution => less quantization error



PAROAttention: evaluation

Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models

Type	Method	Efficiency	Quality						
		Dense Rate / Bitwidth	Video Quality Metrics			FP Diff. Metrics			
			CLIPSIM $\uparrow$	VQA $\uparrow$	Δ FScore $\downarrow$	FVD-FP16 $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	CosSim $\uparrow$
Quant	-	FP16 Full Attn.	100.0%	0.203	92.53	0.000	0.000	∞	1.000
		RTN (INT8)	QK (INT8), PV (INT8)	0.190	92.09	0.571	0.480	18.88	0.956
		RTN (INT4)	QK (INT4), PV (INT4)	0.184	69.25	3.360	1.446	11.99	0.905
		SageAttn	QK (INT8), PV (FP16)	0.203	92.24	0.131	0.047	29.58	0.997
		SageAttnV2	QK (INT4), PV (FP8)	0.200	88.79	2.460	1.750	24.46	0.979
		PAROAttn (INT8)	QK (INT8), PV (INT8)	0.203	92.57	0.096	0.026	29.01	0.935
		PAROAttn (INT4)	QK (INT4), PV (INT4)	0.200	89.24	0.876	1.382	24.16	0.822
Sparse +Quant	PAROAttn (0.3+INT8)	30% + QK, PV (INT8)	0.201	91.68	0.884	0.533	21.49	0.779	0.976
	PAROAttn (0.5+INT4)	50% + QK, PV (INT4)	0.200	90.42	0.967	1.431	24.34	0.827	0.986

PAROAttention: evaluation

Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models

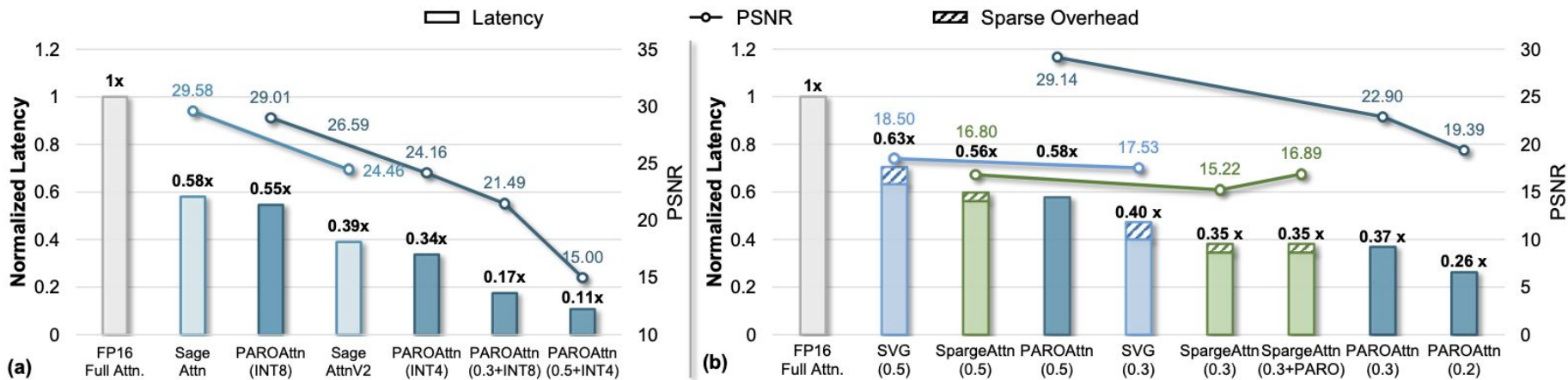


Figure 6: **Normalized latency speedup (bar plot) and PSNR (line plot) trade-off** under different (a) quantization and (b) sparse configurations.

Conclusion

- Attention remains the bottleneck in video diffusion models.
- Attention quantization is challenging.
- Visual-generation-specific structure may be used to help quantization.